In an IT system that manages a ride-sharing platform, it is necessary to efficiently implement:

- the fast search for the “closest” available driver to a new user requesting a ride;
- the dynamic insertion, removal, and update of available drivers as they move or finish a ride;
- the management of a waiting list of requests when no driver is immediately available.

Which data structures and algorithms would you cover in an Algorithms and Data Structures course to design an efficient solution to this problem?

In particular:

- indicate which data structures you would use to maintain and update the set of available drivers;
- explain how you would manage pending requests;
- describe the type of algorithm you would use to assign a driver to a request (e.g., a greedy approach or one based on searching balanced structures);
- analyze the complexity of the fundamental operations (insertion, search, update, priority extraction);
- briefly compare your solution with at least one alternative, less efficient approach.

Justify each choice in light of the topics studied in the course.

Here is a possible **exam-style answer**, coherent with the syllabus of the Algorithms and Data Structures course, which explicitly includes **queues, heaps, hash tables, binary search trees, AVL trees, red-black trees, greedy algorithms, online algorithms, and asymptotic complexity analysis**.

Possible solution

To design an efficient ride-sharing platform, I would use a combination of:

- a **hash table** to partition drivers by geographic area,
- a **balanced binary search tree** (AVL or red-black tree) inside each area to maintain the set of available drivers dynamically,
- and a **queue** or **heap-based priority queue** to manage pending ride requests.

1. Data structures for available drivers

A good course-level solution is to divide the city into **cells or zones**. Then:

- use a **hash table** that maps each zone to the set of currently available drivers in that zone;
- for each zone, store drivers in a **balanced BST** such as an **AVL tree** or a **red-black tree**;
- optionally, maintain another hash table from driver_id to the corresponding node or zone, so that updates and removals are faster.

This solution is realistic and fully aligned with the course contents, since hash tables, AVL trees, and red-black trees are explicitly part of the syllabus.

Why this works

The hash table gives fast access to the relevant geographic bucket, while the balanced tree supports efficient **insertion, deletion, and search** even when the set of drivers changes continuously. Since drivers move frequently, a balanced tree is preferable to an ordinary binary search tree because it guarantees logarithmic height in the worst case. The syllabus explicitly covers AVL trees and red-black trees exactly for this kind of dynamic set maintenance.

2. Management of pending requests

Pending requests can be managed in two possible ways:

- with a **FIFO queue**, if the platform wants a fair “first come, first served” policy;
- with a **priority queue implemented as a heap**, if requests must be ordered by urgency, premium level, or waiting time.

Since the course includes both **queues** and **heaps**, both answers are valid depending on the policy assumed. A very strong exam answer is:

- use a **queue** in the simplest version;
- use a **heap** in the more advanced version, because it allows fast extraction of the highest-priority waiting request.

3. Driver assignment algorithm

The assignment can be described as a **greedy online algorithm**.

When a new request arrives:

1. compute the geographic cell of the user;
2. inspect the balanced structure associated with that cell;
3. if no suitable driver is found, expand the search to neighboring cells;
4. among the candidate available drivers, choose the one with minimum estimated distance;
5. assign that driver immediately and remove them from the set of available drivers.

This is a **greedy approach** because at each step the system chooses the locally best option, namely the currently closest available driver. It is also an **online algorithm**, because requests arrive over time and must be handled without knowing future requests. Both greedy methods and online algorithms are explicitly mentioned in the syllabus.

A balanced-search-based approach is therefore combined with a greedy decision rule: the balanced structure allows efficient maintenance of candidates, while the greedy criterion decides which driver to assign.

4. Complexity analysis

Let:

- (m) be the number of available drivers in one zone,
- (r) the number of pending requests,
- (k) the number of candidate drivers examined during the local search.

Insertion of a driver

When a driver becomes available:

- find the correct cell in the hash table: average **O(1)**
- insert into the AVL/red-black tree of that cell: **O(log m)**

So the total cost is:

- **O(log m)** on average

Removal of a driver

When a driver is assigned or goes offline:

- access the correct cell, then delete from the balanced tree: **O(log m)**

So removal is:

- **O(log m)**

Update of a driver

If the driver moves but remains in the same cell, only a local update may be needed.
If the driver moves to another cell:

- delete from the old balanced tree: **O(log m)**
- insert into the new balanced tree: **O(log m)**

Therefore update is:

- **O(log m)**

Search for the closest driver

- compute the user cell: **O(1)**
- inspect the local tree and possibly neighboring zones
- evaluate candidate distances

A reasonable exam-level complexity is:

- **O(log m + k)**

where (k) is the number of candidate drivers checked. In practice this is much better than scanning all drivers, because the search is localized. In the worst case, if many zones must be explored, it may degrade toward **O(n)**, but usually it remains much smaller than a full scan.

Insertion of a pending request

- with a FIFO queue: **O(1)**

- with a heap priority queue: **O(log r)**

Extraction of the next pending request

- from a FIFO queue: **O(1)**
- from a heap: **O(log r)**

These complexity results are exactly the kind of asymptotic analysis required by the course.

5. Comparison with a less efficient approach

A simpler but less efficient solution would be to store all available drivers in a single **array** or **linked list**.

In that case:

- insertion may be **O(1)**,
- but searching for the closest driver requires scanning all available drivers, so it is **O(n)** for every request,
- deletion or update may also become costly, especially in arrays.

This is clearly worse when the number of drivers is large and requests are frequent. The proposed solution with hash tables and balanced trees scales much better, because it reduces the search space to a local area and guarantees logarithmic update costs.

So, compared with a linear structure:

- the **naive approach** is simpler but has **linear search time**;
- the **balanced and partitioned approach** is more sophisticated but supports efficient dynamic updates and much faster practical nearest-driver search.

Conclusion

In conclusion, an efficient solution within the scope of an Algorithms and Data Structures course is:

- **available drivers**: a **hash table of geographic cells**, each containing an **AVL tree** or **red-black tree** of free drivers;
- **pending requests**: a **FIFO queue** or, if priorities matter, a **heap-based priority queue**;
- **assignment strategy**: a **greedy online algorithm** that searches first in the user's cell and then in neighboring cells, assigning the currently closest available driver;
- **main complexities**:

- insertion/removal/update of drivers: **O(log m)**
- pending request insertion: **O(1)** with queue, **O(log r)** with heap
- request extraction: **O(1)** with queue, **O(log r)** with heap
- closest-driver search: typically **O(log m + k)**, much better in practice than a full **O(n)** scan.

Great idea — having the code in **English with clear comments** makes it perfect for an exam or report.

Below you'll find the **same solution rewritten in English**, clean and well-commented, aligned with Algorithms & Data Structures concepts.

🚗 Program 1 — Ride Sharing System (FIFO Waiting Queue)

```
from dataclasses import dataclass
from collections import defaultdict, deque
import math


@dataclass
class Driver:
    """
    Represents a driver in the system.
    """
    driver_id: int
    x: float
    y: float
    available: bool = True


@dataclass
class RideRequest:
    """
    Represents a ride request made by a user.
    """
    request_id: int
    user_x: float
```

```
    user_y: float


class RideSharingSystemFIFO:
    """
    Ride-sharing system using:
    - Hash table (dict) for drivers
    - Spatial hashing (grid cells) for fast local search
    - FIFO queue (deque) for pending requests

    Algorithmic idea:
    - The city is divided into square cells
    - Each cell stores available drivers
    - Nearest driver is found by expanding search radius
    """

    def __init__(self, cell_size=10):
        self.cell_size = cell_size

        # Hash table: driver_id -> Driver object
        self.drivers = {}

        # Hash table: (cell_x, cell_y) -> set of available driver IDs
        self.available_cells = defaultdict(set)

        # FIFO queue for pending ride requests
        self.waiting_requests = deque()

    def _get_cell(self, x, y):
```

```
    """
    Compute the grid cell for a given position.
    """
    return (int(x // self.cell_size), int(y // self.cell_size))

def _distance(self, x1, y1, x2, y2):
    """
    Compute Euclidean distance between two points.
    """
    return math.hypot(x1 - x2, y1 - y2)

def add_driver(self, driver_id, x, y):
    """
    Insert a new available driver.

    Complexity:
    - O(1) average for dictionary insertion
    - O(1) average for set insertion
    """
    driver = Driver(driver_id, x, y, True)
    self.drivers[driver_id] = driver

    cell = self._get_cell(x, y)
    self.available_cells[cell].add(driver_id)

def remove_driver(self, driver_id):
    """
    Remove a driver from the system.
    """
```

```
    if driver_id not in self.drivers:
        return

    driver = self.drivers[driver_id]

    if driver.available:
        cell = self._get_cell(driver.x, driver.y)
        self.available_cells[cell].discard(driver_id)

    del self.drivers[driver_id]

def update_driver_position(self, driver_id, new_x, new_y):
    """
    Update driver position.
    If the driver changes cell, update the spatial structure.
    """
    if driver_id not in self.drivers:
        return

    driver = self.drivers[driver_id]

    old_cell = self._get_cell(driver.x, driver.y)
    new_cell = self._get_cell(new_x, new_y)

    if driver.available and old_cell != new_cell:
        self.available_cells[old_cell].discard(driver_id)
        self.available_cells[new_cell].add(driver_id)

    driver.x = new_x
```

```
    driver.y = new_y

def set_driver_available(self, driver_id, available=True):
    """
    Change driver availability status.
    """
    if driver_id not in self.drivers:
        return

    driver = self.drivers[driver_id]
    cell = self._get_cell(driver.x, driver.y)

    # Driver becomes available
    if available and not driver.available:
        driver.available = True
        self.available_cells[cell].add(driver_id)

        # Try to serve a pending request
        self._try_assign_waiting_request()

    # Driver becomes unavailable
    elif not available and driver.available:
        driver.available = False
        self.available_cells[cell].discard(driver_id)

def find_closest_driver(self, user_x, user_y, max_radius=5):
    """
    Greedy nearest-driver search.
```

```
Strategy:
- Start from user's cell
- Expand search to neighboring cells
- Return the closest available driver found
"""
start_cell = self._get_cell(user_x, user_y)

best_driver_id = None
best_distance = float("inf")

for r in range(max_radius + 1):
  cx, cy = start_cell

  for dx in range(-r, r + 1):
    for dy in range(-r, r + 1):
      cell = (cx + dx, cy + dy)

      for driver_id in self.available_cells[cell]:
        driver = self.drivers[driver_id]
        dist = self._distance(user_x, user_y, driver.x, driver.y)

        if dist < best_distance:
          best_distance = dist
          best_driver_id = driver_id

  # Greedy stop: return first valid local solution
  if best_driver_id is not None:
    return best_driver_id, best_distance
```

```
    return None, None

def request_ride(self, request_id, user_x, user_y):
    """
    Handle a new ride request.
    """
    driver_id, dist = self.find_closest_driver(user_x, user_y)

    if driver_id is not None:
        self.set_driver_available(driver_id, False)

        return {
            "status": "assigned",
            "request_id": request_id,
            "driver_id": driver_id,
            "distance": dist
        }

    # No driver available → enqueue request
    request = RideRequest(request_id, user_x, user_y)
    self.waiting_requests.append(request)

    return {
        "status": "queued",
        "request_id": request_id
    }

def _try_assign_waiting_request(self):
    """
```

```
    Try to serve the first request in FIFO queue.
    """
    if not self.waiting_requests:
        return None

    request = self.waiting_requests[0]
    driver_id, dist = self.find_closest_driver(request.user_x, request.user_y)

    if driver_id is None:
        return None

    self.waiting_requests.popleft()
    self.set_driver_available(driver_id, False)

    return {
        "status": "assigned_from_queue",
        "request_id": request.request_id,
        "driver_id": driver_id,
        "distance": dist
    }
```

⚡ **Program 2 — Version with Priority Queue (Heap)**

```
from dataclasses import dataclass, field
from collections import defaultdict
import heapq
import math
```

```
@dataclass
class Driver:
    driver_id: int
    x: float
    y: float
    available: bool = True


@dataclass(order=True)
class PrioritizedRequest:
    """
    Priority queue element.
    Lower value = higher priority.
    """
    priority: int
    request_id: int = field(compare=False)
    user_x: float = field(compare=False)
    user_y: float = field(compare=False)


class RideSharingSystemPriority:
    """
    Ride-sharing system with priority queue for pending requests.
    """

    def __init__(self, cell_size=10):
        self.cell_size = cell_size
        self.drivers = {}
        self.available_cells = defaultdict(set)
```

```
    # Min-heap for pending requests
    self.waiting_requests = []

def _get_cell(self, x, y):
    return (int(x // self.cell_size), int(y // self.cell_size))

def _distance(self, x1, y1, x2, y2):
    return math.hypot(x1 - x2, y1 - y2)

def add_driver(self, driver_id, x, y):
    driver = Driver(driver_id, x, y, True)
    self.drivers[driver_id] = driver
    self.available_cells[self._get_cell(x, y)].add(driver_id)

def set_driver_available(self, driver_id, available=True):
    if driver_id not in self.drivers:
        return

    driver = self.drivers[driver_id]
    cell = self._get_cell(driver.x, driver.y)

    if available and not driver.available:
        driver.available = True
        self.available_cells[cell].add(driver_id)
        self._try_assign_waiting_request()

    elif not available and driver.available:
        driver.available = False
```

```
        self.available_cells[cell].discard(driver_id)

    def find_closest_driver(self, user_x, user_y, max_radius=5):
        start_cell = self._get_cell(user_x, user_y)

        best_driver_id = None
        best_distance = float("inf")

        for r in range(max_radius + 1):
            cx, cy = start_cell

            for dx in range(-r, r + 1):
                for dy in range(-r, r + 1):
                    for driver_id in self.available_cells[(cx + dx, cy + dy)]:
                        driver = self.drivers[driver_id]
                        dist = self._distance(user_x, user_y, driver.x, driver.y)

                        if dist < best_distance:
                            best_distance = dist
                            best_driver_id = driver_id

            if best_driver_id is not None:
                return best_driver_id, best_distance

        return None, None

    def request_ride(self, request_id, user_x, user_y, priority=10):
        """
        Insert request into priority queue if no driver is available.
```

```
    """
    driver_id, dist = self.find_closest_driver(user_x, user_y)

    if driver_id is not None:
        self.set_driver_available(driver_id, False)
        return {"status": "assigned", "driver_id": driver_id}

    heapq.heappush(
        self.waiting_requests,
        PrioritizedRequest(priority, request_id, user_x, user_y)
    )

    return {"status": "queued_with_priority"}

def _try_assign_waiting_request(self):
    """
    Extract highest-priority request and assign driver.
    """
    if not self.waiting_requests:
        return None

    request = self.waiting_requests[0]
    driver_id, dist = self.find_closest_driver(request.user_x, request.user_y)

    if driver_id is None:
        return None

    heapq.heappop(self.waiting_requests)
    self.set_driver_available(driver_id, False)
```

```
    return {
        "status": "assigned_from_priority_queue",
        "request_id": request.request_id,
        "driver_id": driver_id
    }
```

🧠 **Final note (useful for exam)**

You can explicitly state:

- dict → **Hash Table (O(1) average)**
- deque → **Queue (O(1))**
- heapq → **Priority Queue (O(log n))**
- spatial grid → **reduces search space**
- algorithm → **Greedy + Online**

If you want, I can also:

- add a **true AVL tree implementation** (very exam-oriented 🔥)
- or convert this into a **short 15-line exam answer + pseudocode**